

1 Le processeur et les registres

1.1 Registres et sous-registres

Exercice 1. Quelles sont les valeurs minimales et maximales qui peuvent être enregistrées respectivement dans les registres eax, ax, ah et al ? Les valeurs données doivent être exprimées en décimal non signé, en décimal signé et en hexadécimal (non signé).

	eax (32 bits)	ax (16 bits)	ah (8 bits)	al (8 bits)
Unsigned	- Min = 0 - Max = 2^{32}	- Min = 0 - Max = $2^{16} - 1$	- Min = - Max =	- Min = - Max =
Signed	- Min = - Max =	- Min = - Max =	- Min = - Max =	- Min = - Max =
Hexa	- Min = - Max =	- Min = - Max =	- Min = - Max =	- Min = - Max =

- **eax (32bits)** : non signé (min=0 et max= 2^{32}), signée (min= $-(2^{31} - 1)$ et max= $2^{32}-1$) et hexadécimal (min=0x00000000 et max=0xFFFFFFFF).
- **ax (16bits)** : non signé (min=0 et max= 2^{16}), signée (min= $-(2^{16}-1) - 1$) et max= $2^{16}-1$) et hexadécimal (min=0x0000 et max=0xFFFF) => car chaque lettre = 4bits.
- **ah (8bits)** : non signé (min=0 et max= 2^8), signée (min= $-(2^{8-1} - 1)$ et max= $2^{8-1} - 1$) et hexadécimal (min=0x00et max=0xFF) => car chaque lettre = 4bits.
- **al (8bits)** : non signé (min=0 et max= 2^8), signée (min= $-(2^{8-1} - 1)$ et max= $2^{8-1} - 1$) et hexadécimal (min=0x00et max=0xFF) => car chaque lettre = 4bits.

Registre EAX (32 bits) :

- Non signé : Min = 0, Max = $2^{32}-1$
- Signé : Min = -2^{31} , Max = $2^{32}-1$
- Hexadécimal : Min = 0x00000000, Max = 0xFFFFFFFF

Registre AX (16 bits) :

- Non signé : Min = 0, Max = $2^{16}-1$
- Signé : Min = -2^{15} , Max = $2^{15}-1$
- Hexadécimal : Min = 0x0000, Max = 0xFFFF

Registres AH/AL (8 bits) :

- Non signé : Min = 0, Max = 2^8-1
- Signé : Min = -2^7 (soit -128) , Max = 2^7-1 (soit 127)
- Hexadécimal : Min = 0x00, Max = 0xFF

1.2 Opérations sur les registres

Exercice 2. Quelles sont les valeurs (en hexadécimal) de eax, ax, ah et al après l'instruction `mov eax, 134512768` ? Quelles sont ensuite les valeurs (en hexadécimal) de eax, ax, ah et al après l'instruction `mov al, 0` ?

(134512768)déc = (08048080)hex

Donc eax vaut 0x08048080, ax vaut 0x8080, ah et al vaut 0x80.

Après l'instruction "`mov, al, 0`", eax vaut 0x08048000, ax vaut 0x8000, ah vaut 0x80 et al vaut 0x00.

Exercice 3. ■ En utilisant uniquement l'instruction `mov` et des constantes exprimées en hexadécimal n'occupant pas plus de deux chiffres hexadécimaux, donner une suite d'instructions qui modifie la valeur de ebx de sorte qu'elle soit égale à 0x0000ABCD.

```
mov ebx, 0           ; ebx = ebx + 0
mov bl, 0xCD         ; bl = 0xCD
mov bh, 0xAB         ; bh = 0xAB
```

Exercice 4. Après l'exécution des instructions suivantes, quelles sont les valeurs des registres eax, ax, ah et al ?

```
mov eax, 0xf00000f0
eax vaut 0xf00000f0, ax vaut 0x00f0, ah vaut 0x00 et al vaut 0xf0.
```

```
add ax, 0xf000
eax vaut 0xf000f0f0, ax vaut 0xf0f0, ah vaut 0xf0 et al vaut 0xf0.
```

```
add ah, al
eax vaut 0xf001e0f0, ax vaut 0xe0f0, ah vaut 0xe0 et al vaut 0xf0.
```

1.3 Lecture/écriture en mémoire

Exercice 5. (Mémoire)

1. Combien de valeurs différentes peut représenter une case de la mémoire ?

On sait qu'une case contient un octet soit 8 bits. Donc on peut représenter $2^8 = 256$ valeurs différentes dans une case mémoire.

2. Quelle est la quantité (exprimée en giboctets) de mémoire adressable sur 32 bits ?

Un giboctet = 2^{30} octets et on a un tab de 2^{32} cases. Chaque case contient un octet. On a donc 2^{32} octets. Donc $2^{32} \text{ octets} / 2^{30} \text{ octets} = 2^2$ Gib soit 4 Gib.

3. Combien de cases se situent avant la case d'indice 0x0000100a dans la mémoire ?

On a 100a hexa => en binaire on : 0001 0000 0000 1010 soit 4106 donc on a 4105 cases situé avant la case d'indice 0x0000100a dans la mémoire.

Donc, il y a 4106 cases avant 0x100a (si on considère que la première case est à l'indice 0).

1.3.1 Lecture en mémoire

Exercice 6. Expliquer la différence entre `mov eax, 3` et `mov eax, [3]`.

`mov eax, 3` => `eax` reçoit la valeur 3.

`mov eax, [3]` => `eax` reçoit la donnée stockée à l'adresse 3.

Exercice 7. Dans l'exemple de mémoire de la table 2, donner les valeurs des registres ou sous-registres demandés, après les instructions suivantes (avant chaque sous-question, `eax` est supposé égal à 0x0) :

1. `ax` après l'instruction `mov ax, [1]` ;
`ax` (0x5A1E).
2. `ah` après l'instruction `mov ah, [0]` ;
`ah` (0x03).
3. `eax` après l'instruction `mov eax, [0]` ;
`eax` (0x0A5A1E03).
4. `eax` après l'instruction `mov eax, [1]`.
`eax` (0x100A5A1E).

Exercice 8.

Quelle est la valeur de `eax` à la fin de la suite d'instructions suivante, en supposant que la mémoire est dans l'état de la table 2 ?

`mov eax, 5`

`eax` => 5 0x00000005

`sub eax, 1`

`eax` => 4 0x00000004

`mov al, [eax]`

`eax` => 16 0x00000010

mov ah, [eax]

eax => 0x00007F10

Exercice 9. Quel est l'état de la mémoire après l'exécution de la suite d'instructions qui suit?

mov eax, 0x04030201 => dans eax on écrit la val 0x04030201 soit 4 octets où eax = 04030201, ax = 0201, ah = 02 et al = 01. (voir schéma brouillon)

mov [0], eax => en mémoire (petit boutisme) on a adr0 = 0x01, adr1=0x02, adr2=0x03 et adr3=0x04.

mov [2], ax => en mémoire (petit boutisme) on a adr2=0x01 et adr3=0x02.

mov [3], al => en mémoire (petit boutisme) on a adr2=0x01 et adr3=0x01.

L'état final de la mémoire (petit boutisme) est adr0 = 0x01, adr1=0x02, adr2=0x01 et adr3=0x01.

Exercice 10. Quel est l'état de la mémoire après avoir effectué la suite d'instructions suivante ?

mov dword [0], 0x020001 => en mémoire (petit boutisme) on a adr0 = 0x01, adr1=0x00, adr2=0x02 et adr3=0x00 car dword = 4 octets et on a un octet par case.

mov byte [1], 0x21 => en mémoire (petit boutisme) on a adr0 = 0x01, adr1=0x21, adr2=0x02 et adr3=0x00 car byte = 1 octets et on a un octet par case.

mov word [2], 0x1 => en mémoire (petit boutisme) on a adr0 = 0x01, adr1=0x21, adr2=0x01 et adr3=0x00 car word = 2 octets et on a un octet par case.

Exercice 11. Quelle est la valeur de **eax** à la fin de la suite d'instructions suivante dans la convention *little-endian* et *big-endian* ?

mov ax, 0x0001 => dans ax on écrit la val 0x0001 soit 2 octets où ax = 0001, ah = 00 et al = 01.

mov [0], ax => en mémoire (petit boutisme) on a adr0 = 0x01 et adr1=0x00 ET (grand boutisme) on a adr0 = 0x00 et adr1=0x01.

mov eax, 0 => dans eax on écrit la val 0x00000000 soit 4 octets où ax = 0000, ah = 00 et al = 00.

mov al, [0] => dans al on écrit la val à l'adresse 0 soit 0x01 en petit boutisme et 0x00 en grand boutisme.

A la fin dans la mémoire on a eax = 0x00000001 en little endian et eax = 0x00000000 en big endian.

Exercice 12. Modifier Hello.asm pour afficher « L'ordi va passer en veille ! ». Le nouveau programme doit s'appeler E12.asm.

section .data

msg:

db "L'ordi va passer en veille !", 10 ; 29 caractères

section .text

global main

main:

```
mov edx, 29      ; Longueur de la chaîne
mov ecx, msg     ; Adresse de la chaîne
mov ebx, 1       ; Sortie standard (stdout)
mov eax, 4       ; Numéro "write" (syscall)
int 0x80         ; Appel système pour afficher
mov ebx, 0       ; Code retour 0
mov eax, 1       ; Numéro "exit" (syscall)
int 0x80         ; Quitter le programme
```

Exercice 13. Commenter les différences (éventuelles) entre les fichiers Hello.lst et E12.lst en les expliquant.

Dans les fichiers .lst :

Section .data : la chaîne stockée (octets codés en hexa) sera différente — plus longue dans E12.lst.

- Hello.lst : 00000000 48656C6C6F20576F72-6C640A
- E12.lst : (par exemple) 00000000 4C276F726469207661-2070617373657220-6565207665696C6C6520210A

Section .text :

- L'instruction mov edx, ... (qui encode la longueur de la chaîne) aura une valeur différente.
- La longueur diffère : Hello.asm utilise 12 (mov edx, 0xc), E12.asm utilisera 27 (mov edx, 0x1b si 27 caractères).
- Instruction mov ecx, msg : la constante ajoutée à la machine finale pour l'adresse du message sera différente car la section .data sera plus grande (mais c'est transparent pour l'utilisateur).

Explications :

- La seule différence fonctionnelle : taille de la chaîne en mémoire (+ le nombre stocké dans edx).
- Les instructions assemblées (mov edx, ..., mov ecx, msg) utiliseront la nouvelle taille/adresse.
- Cela se reflète dans les codes d'opcodes générés dans le .lst (même instructions sinon).

Exercice 14. Écrire un programme E14.asm respectant les consignes suivantes. Dans la section de données du programme, il doit être défini une étiquette debut sur la chaîne de caractères « Le deuxieme TP » ainsi qu'une étiquette fin sur la chaîne de caractères « est presque termine ! ». Par deux appels système, le programme doit afficher la chaîne adressée par debut puis celle adressée par fin. La terminaison du programme doit être gérée de manière propre.

section .data

debut: db "Le deuxieme TP", 10 ; 15 caractères + \n

fin: db "est presque termine !", 10 ; 22 caractères + \n

section .text

global main

main:

; Affiche la 1e chaine

mov edx, 15 ; longueur "Le deuxieme TP\n"

mov ecx, debut ; adresse

mov ebx, 1 ; stdout

mov eax, 4 ; syscall write

int 0x80

; Affiche la 2e chaine

mov edx, 23 ; longueur "est presque termine !\n"

mov ecx, fin ; adresse

mov ebx, 1

mov eax, 4

int 0x80

; Quitter proprement

mov ebx, 0 ; code retour

mov eax, 1 ; syscall exit

int 0x80

Exercice 15. Fournir le listing E14.lst du programme E14.asm et l'étudier pour calculer la taille du code machine du programme. Le raisonnement doit apparaître dans la réponse.